

Arquitetura do RSA_APB

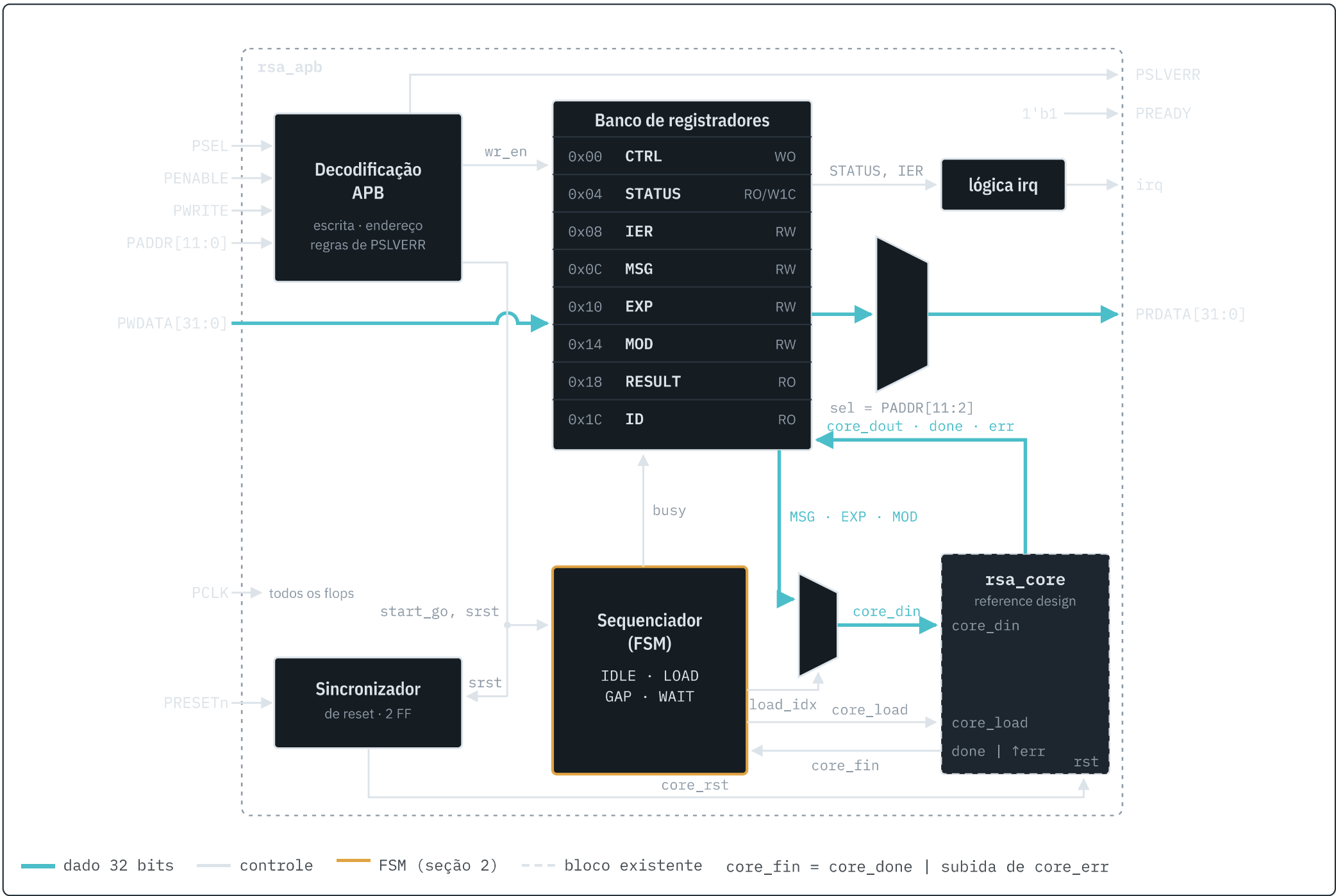
Referência para a etapa de desenho: diagrama de blocos, máquina de estados do sequenciador e mapa de registradores. Copie os três no papel antes de abrir o editor; o RTL sai quase como transcrição.

ID 0x5253_0200 · "RS" v2.0 · C = M e mod n

- 1
- Diagrama de blocos
- 2
- FSM do sequenciador
- 3
- Mapa de registradores

Diagrama de blocos 1 / 3

O wrapper cerca o `rsa_core` com cinco partes. Os barramentos de 32 bits (em cor) mostram o caminho do dado: `PWDATA` entra nos registradores, os operandos descem até o core, o resultado volta para `RESULT` e sai por `PRDATA`.



O que cada bloco faz

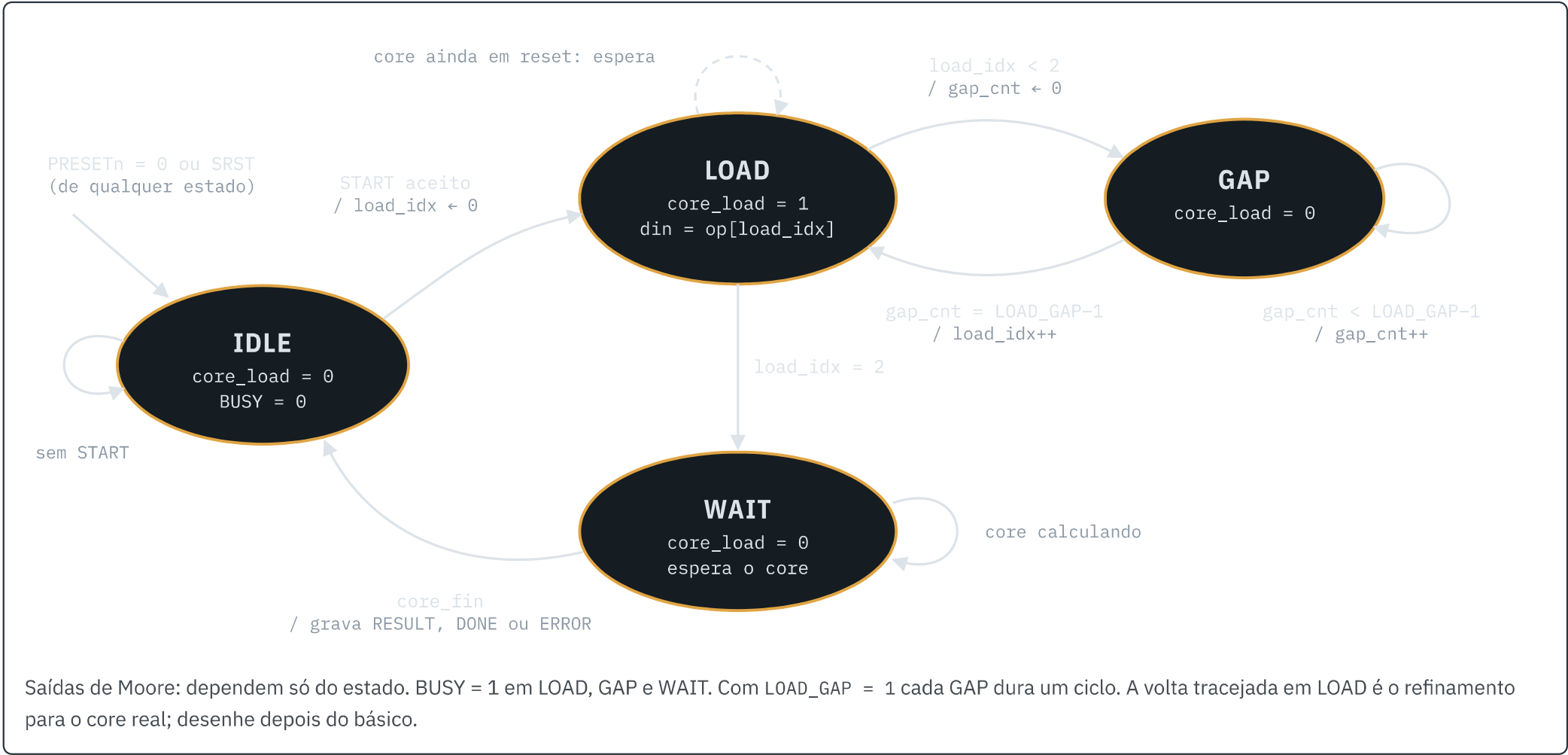
- Decodificação APB:** escrita quando `PSEL` & `PENABLE` & `PWRITE`; escolhe o registrador por `PADDR[11:2]`; gera `PSLVERR`.
- Banco de registradores:** guarda operandos, flags e resultado.
- Mux de PRDATA:** numa leitura, coloca o registrador endereçado na saída.
- Sequenciador:** entrega M, e, n ao core e espera o fim (seção 2).
- Sincronizador de reset:** `core_rst = ~rst_sync[1] | srst_q`.

Para conferir no seu desenho

- ☐ As 11 portas da tabela 3.1 da spec, com nome e largura
- ☐ `PREADY` amarrado em 1
- ☐ Os 3 caminhos do dado: entrada, operandos, resultado
- ☐ As 3 entradas da FSM: `start_go`, `srst`, `core_fin`
- ☐ `busy` voltando da FSM para o `STATUS`

FSM do sequenciador 2 / 3

Quatro estados e dois contadores. load_idx diz qual operando sai (0 = M, 1 = e, 2 = n) e gap_cnt conta os ciclos de intervalo. Nas setas, a condição vem antes da barra e a ação nos contadores depois dela.



Exemplo no tempo: 65¹⁷ mod 3233, LOAD_GAP = 1

CICLO	ESTADO	LOAD_IDX	CORE_LOAD	CORE_DIN	BUSY	O QUE ACONTECE
0	IDLE → LOAD	0	0	—	0 → 1	Escrita do START aceita; DONE e ERROR limpos
1	LOAD	0	1	65	1	Entrega M
2	GAP	0	0	—	1	Intervalo
3	LOAD	1	1	17	1	Entrega e
4	GAP	1	0	—	1	Intervalo
5	LOAD → WAIT	2	1	3233	1	Entrega n, o último
6 – 3175	WAIT	2	0	—	1	Core calculando
3176	WAIT → IDLE	—	0	—	1	core_done com core_dout = 2790
3177	IDLE	—	0	—	0	RESULT = 2790, DONE = 1

Mapa de registradores 3 / 3

Oito registradores de 32 bits a partir do endereço base. Bits não listados são reservados: leem 0. Offsets de 0x20 a 0xFFC são reservados.

OFFSET	NOME	ACESSO	RESET	CAMPOS	COMPORTAMENTO
0x00	CTRL	WO	0x0000_0000	[1] SRST [0] START	Bits auto-limpantes. Leitura retorna 0. SRST vence START na mesma escrita.
0x04	STATUS	RO / W1C	0x0000_0000	[2] ERROR [1] DONE [0] BUSY	DONE e ERROR ficam em 1 até o software escrever 1 no bit. START também limpa. BUSY é só leitura.
0x08	IER	RW	0x0000_0000	[1] ERR_IE [0] DONE_IE	Habilita cada flag a ligar irq.
0x0C	MSG	RW	0x0000_0000	[31:0] M	Escrita em qualquer ordem; valor mantido entre operações. Escrita com BUSY = 1 dá PSLVERR e não altera.
0x10	ERR	RW	0x0000_0000	[31:0] E	Indica erro de acesso a qualquer registrador. Escrita com BUSY = 1 dá PSLVERR e não altera.

Arquitetura RSA_APB					
0x10	EXP	RW	0x0000_0000	[31:0] e	
0x14	MOD	RW	0x0000_0000	[31:0] n	
0x18	RESULT	RO	0x0000_0000	[31:0] C	Capturado no fim com sucesso; inalterado em erro e em SRST.
0x1C	ID	RO	0x5253_0200	[31:16] "RS" [15:8] 0x02 [7:0] 0x00	Identificador fixo do bloco e versão.

Campos de bits

CTRL	31:2 reservado	1 SRST	0 START
STATUS	31:3 reservado	2 ERROR	1 DONE
			0 BUSY
IER	31:2 reservado	1 ERR_IE	0 DONE_IE
ID	31:16 0x5253 "RS"	15:8 0x02	7:0 0x00

Quando PSLVERR = 1

- Escrita em RESULT ou ID
 - Escrita em MSG, EXP ou MOD com BUSY = 1
 - Escrita em CTRL com START = 1 (e SRST = 0) com BUSY = 1
- Só na fase de acesso. Leituras e endereços reservados nunca dão erro.

Equações de uma linha

- escrita = PSEL & PENABLE & PWRITE
- irq = DONE&DONE_IE | ERROR&ERR_IE
- BUSY = (state != IDLE)
- PREADY = 1